



CSE471: System Analysis and Design
Assignment on Functional Requirements
Proposed Project Title: Tryora

Group No: 08, CSE471 Lab Section: 01, Spring 2026		
SL	ID	Name
1	22299082	Hirok Roy Rahul
2	22299059	Toha Nahin
3	22101463	Md. Mustafizur Rahman Minhaz
4	22101453	Abdullah Al Jubayer

Submission Date:

Project Overview

A web application that lets users create a personalized 3D avatar from full-body photos and dress it according to an event-based wardrobe system.

Tech Stack:

- Language: Typescript, JavaScript, Python
- Framework: Next.js 16, FastAPI, Express.js
- Styling: TailwindCSS
- 3D Engine: Three.js (@react-three/fiber)
- Database: PostgreSQL (Relational DB), Qdrant (Vector DB), Redis (key-value cache)
- ORM: Prisma, SQLAlchemy
- Deployment: Vercel (Frontend), Railway (API), Vast.ai (GPU Worker)
- External APIs:
 - **Ready Player Me API:** Avatar creation
 - **Nodemailer API:** For Sending Notifications and Reminders.
 - **Cloudflare R2 & Uploadthings API:** Storage
 - **OpenClawBot ApI:** Web Scraping
 - **OpenAI/ Google Gemini API:** LLM Reasoning

User Roles

1. **User:** Primary role; can upload photos for avatar generation, browse scraped wardrobes, customize outfits, and export renders.
2. **Scraping Worker (System):** An automated background role that crawls fashion sites, parses product metadata, and updates the vector database.
3. **Administrator:** Responsible for monitoring system performance, managing the site adapter allowlist for scraping, and auditing robots.txt compliance.

Functional Requirements

SL	Common Workflows	Corner Case
1	User Authentication & session management: Users can create an account by providing basic details such as name, email, phone number, and password.	Users can create an account by providing basic details such as name, email, phone number, and password.
2	Market Discovery & Search: The system automatically scrapes local fashion sites (like openclaw.ai) and stores products with semantic embeddings. Users can search for items using natural language (e.g., "red dress for Eid")	The system automatically scrapes local fashion sites (like openclaw.ai) and stores products with semantic embeddings. Users can search for items using natural language (e.g., 'red dress for Eid').

Module 1		
Member	Feature Description	Corner Case
1	Photo Upload & 3D Avatar Generation: Extracts 33 body landmarks in-browser via MediaPipe to create a proportional 3D avatar. Includes a manual slider fallback for users with low-quality photos	MediaPipe fails silently on dark/blurry photos and returns partial landmark data, producing a distorted avatar. → Set a minimum confidence threshold (0.7) for all 33 landmarks; if any are below the threshold, automatically activate the manual slider fallback and surface a clear prompt to the user.
2	Market Parsing Dress Discovery: Develops the Playwright-powered scraping service for myclaw.ai to extract titles, prices, and images for the database.	myclaw.ai updates its HTML structure overnight, breaking selectors silently — the scraper reports success but returns 0 products. → Assert a minimum product count per run (e.g., ≥ 5 items); auto-pause the adapter and alert the administrator if the count falls below the threshold.
3	Budget Filter & Price Display: Creates the price-range slider and "Best Value"	A price range like BDT 0–100 returns an empty result set with no user feedback, leaving the page blank.

	badge logic to filter products by BDT cost tiers.	→ Detect empty filter results before rendering; show a 'No items in this range — try widening your budget' message with a one-click reset to the default range.
4	Trending & Popular Items Carousel: Builds the Redis-backed tracking system to display the most-viewed and most-saved items at the top of the wardrobe.	Redis pod restart flushes view/save counts, causing the carousel to show stale or random items until enough new interactions accumulate. → Persist a daily snapshot of trending scores to PostgreSQL; on Redis miss or flush, seed Redis from the last PostgreSQL snapshot rather than starting from zero.

Module 2		
Member	Feature Description	Corner Case
1	Real-Time Dress Fitting: Develops the 3D mesh overlay system to pin dress models to the avatar skeleton using bone parenting and morph targets.	Bone parenting breaks when the avatar's landmark ratios fall outside the dress mesh's expected range (very tall/short users), causing extreme mesh distortion. → Clamp all bone scale factors to a $\pm 30\%$ deviation from the mesh's neutral pose; apply a smoothing pass on morph target weights to prevent sudden geometry spikes.
2	Body Measurement Based Size Recommendation: Maps MediaPipe landmark ratios to brand-specific size charts to suggest S/M/L/XL fits.	A brand's size chart data is missing or has inconsistent units (cm vs. inches), causing the recommendation engine to suggest wildly incorrect sizes. → Validate every imported size chart against a canonical schema; normalize all units to cm on ingestion and flag charts with missing fields for administrator review before enabling them.
3	Event-Based Filtering & Recommendations: Implements the context-aware selector (Wedding, Eid, etc.)	An event category (e.g., 'Pohela Boishakh') has no scraped products tagged for it yet, returning a silently empty recommendation list.

	that re-ranks products based on event suitability.	→ Maintain a coverage map per event category in PostgreSQL; surface a 'More styles coming soon' placeholder for sparse categories rather than an empty page, and trigger a targeted scrape job.
4	Color & Pattern Variant Switcher: Creates the UI and logic to swap material textures on the 3D model when different color variants are selected.	A texture swap is triggered before the new texture file has fully loaded from R2, causing a brief flash of the raw grey mesh visible to the user. → Preload the next texture into an offscreen canvas before committing the swap; only update the visible material reference once the texture's onLoad callback fires.

Module 3		
Member	Feature Description	Corner Case
1	Interactive Body-Part Selection: Implements 3D raycasting zones on the avatar, allowing users to click body parts to browse specific accessories.	On mobile, touch events fire at lower precision than mouse events, causing raycasts to miss small zones (hands, neck) and select the wrong body part. → Enlarge raycasting hit-sphere radii by 40% on touch devices (detected via pointer type); add a visual highlight ring on hover/tap to confirm the selected zone before opening the accessory panel.
1	Render Export High-Quality Images: Builds the WebGL canvas capture system to generate 2K social-ready images with custom backdrops.	Export is triggered before all textures finish loading, producing a PNG with a grey placeholder mesh instead of the dressed avatar. → Gate the export button behind a scene-ready boolean that turns true only after all pending texture onLoad callbacks have fired; show a spinner until ready.
2	Outfit Save & Wishlist: Develops the PostgreSQL storage logic to save full outfit configurations, including avatar parameters and applied items.	A user saves an outfit that references a product URL, which is later delisted by the scraper, leaving a broken reference in the saved configuration.

		→ Store a snapshot of the product's title, image URL, and price at save time alongside the live product ID; display the snapshot data if the live product is no longer found.
2	Side-by-Side Outfit Comparison: Provides a split-screen view rendering two independent 3D contexts to compare price, event suitability, and style between two different outfits	Initializing two WebGL contexts simultaneously on mobile exceeds GPU memory limits, causing one or both contexts to silently fail to render. → Detect available GPU memory via the WEBGL_debug_renderer_info extension; if below a safe threshold, render the two outfits sequentially in a single context with a toggle switch instead of true split-screen.
3	Outfit Share via Link: Creates the read-only sharing route and Open Graph metadata generator for rich previews on WhatsApp/social media.	Switching between animation clips mid-playback without blending causes the avatar mesh to snap abruptly, visually breaking the silhouette. → Use Three.js AnimationMixer's crossFadeTo() with a 0.3-second blend duration when switching clips; disable the pose selector buttons during the blend window to prevent rapid re-triggering.
3	Avatar Pose Selector: Integrates preset animation clips (walking, sitting, standing) so users can preview the outfit in motion.	Switching between animation clips mid-playback without blending causes the avatar mesh to snap abruptly, visually breaking the silhouette. → Use Three.js AnimationMixer's crossFadeTo() with a 0.3-second blend duration when switching clips; disable the pose selector buttons during the blend window to prevent rapid re-triggering.
4	Offline Mode Cached Wardrobe: Sets up the Service Worker (PWA-lite) to cache the catalog and saved outfits for use without an internet connection.	The Service Worker caches a stale version of the wardrobe catalog after a scraping update, so offline users see outdated product data even when back online. → Version the Service Worker cache key with a build hash and the last scrape timestamp; on

		activation, purge all caches with older keys and broadcast a 'catalog updated' event to open tabs.
4	Accessibility Mode: Ensures WCAG compliance with keyboard navigation, screen reader ARIA labels, and reduced motion settings.	The 3D canvas receives no keyboard focus events, making the entire avatar-fitting interaction completely inaccessible to keyboard-only and screen reader users. → Wrap the canvas in a focusable div with role='application' and a descriptive aria-label; implement keyboard shortcuts (Tab to cycle body parts, Enter to select) and honour prefers-reduced-motion by defaulting to the static pose view.